

# Safe AI-Assisted Self-Adaptation for Kubernetes Microservices

## A Whitepaper on Deterministic Control, Safety Validation, and Explainable Runtime Adaptation

**Project:** Self-Adaptive Microservices

**Prototype:** Kubernetes-based MAPE-K controller with safety-gated execution

**Status:** Research prototype

**Date:** 2026

### Abstract

Modern microservice systems operate under volatile workloads, changing resource availability, and strict service-level objectives. Kubernetes provides standard autoscaling mechanisms such as the Horizontal Pod Autoscaler, but many production systems require adaptation decisions that account for more than CPU or memory utilization. Latency, error rate, throughput, readiness, cooldown behavior, cost, and operator trust all matter. At the same time, the growing interest in AI-assisted operations raises an important safety question: how can intelligent reasoning improve explainability and context awareness without giving probabilistic components direct authority over production infrastructure?

This whitepaper presents a research prototype for safe self-adaptation of Kubernetes-based microservices. The prototype implements a MAPE-K feedback loop that monitors Prometheus metrics, analyzes service health, plans adaptation actions, validates plans through deterministic safety guards, optionally executes approved Kubernetes scaling actions, records outcomes in a knowledge store, and generates explanations through a reasoning layer. The central architectural claim is that AI-supported reasoning should be separated from actuation. The LLM is never the actuator: it may explain, summarize, or contextualize decisions, but deterministic validation and execution components remain responsible for enforcing safety and applying changes.

The prototype is intended for researchers, platform engineers, and software engineering teams who want to study or extend safe adaptive control for cloud-native systems. It can be used as a reference architecture, an experimental framework for comparing adaptive strategies, and a foundation for future production-grade adaptive controllers.

### Keywords

Self-adaptive systems, MAPE-K, Kubernetes, microservices, autoscaling, safety validation, LLMOps, AIOps, Prometheus, explainability, cloud-native systems, autonomic computing.

## 1. Introduction

Cloud-native software systems increasingly rely on microservice architectures deployed on Kubernetes. These systems must respond to changing workload intensity, dependency failures, resource contention, and cost constraints. Manual intervention is often too slow, while simple autoscaling policies may be too narrow to capture the operational realities of complex services.

Kubernetes Horizontal Pod Autoscaler (HPA) provides a widely used baseline for horizontal scaling. HPA is implemented as a Kubernetes API resource and controller that periodically adjusts target scale based on observed metrics, such as CPU utilization, memory utilization, or custom metrics. This model is effective for many workloads, but it does not by itself provide a complete framework for explainable, policy-aware, safety-gated adaptation across multiple operational objectives.

Self-adaptive systems research has long proposed feedback-loop architectures in which software monitors itself, analyzes runtime conditions, plans changes, executes adaptations, and stores knowledge for future decisions. This is commonly described as the MAPE-K model: Monitor, Analyze, Plan, Execute, and Knowledge. Recent advances in large language models create new opportunities for explanation, operator

support, and contextual reasoning. However, they also introduce risks if probabilistic outputs are allowed to directly mutate infrastructure.

This prototype explores a conservative design: combine deterministic control with AI-assisted reasoning, but preserve a strict safety boundary.

The governing principle is:

AI may explain or suggest.

Safety validation must decide what is allowed.

Deterministic executors must apply approved actions.

The knowledge layer must record what happened.

## 2. Problem Statement

Microservice autoscaling and runtime adaptation face several practical challenges:

- **Metric incompleteness:** Observability systems can fail, lag, or return partial data.
- **Narrow scaling signals:** CPU-based scaling may miss latency, error-rate, readiness, or dependency-related problems.
- **Unsafe automation:** A scaling action can worsen instability if it ignores replica limits, cooldown windows, rollout state, or cost budgets.
- **Poor explainability:** Operators need to understand why an automated system proposed, approved, rejected, or executed an action.
- **Reproducibility:** Research and production audits require structured records of metrics, plans, validations, and outcomes.
- **AI safety:** LLMs can be helpful for explanation but should not directly control infrastructure.

The research question behind this prototype is:

Can AI-assisted reasoning improve explainability and context awareness in adaptive microservice control while deterministic safety mechanisms preserve reproducibility and operational safety?

## 3. Design Goals

The prototype is designed around the following goals.

### 3.1 Deterministic Core Control

The monitor, analyzer, planner, safety validator, and executor are deterministic software components. Given the same inputs and configuration, they should produce repeatable results.

### 3.2 Safety-Gated Execution

Every proposed adaptation plan must pass safety validation before execution. Safety checks include replica bounds, total replica budget, estimated cost budget, and cooldown constraints.

### 3.3 Explainability Without Actuator Authority

The reasoning layer can explain decisions and retrieve context, but it cannot directly execute Kubernetes operations or override safety validation.

### 3.4 Reproducible Experimentation

Metrics, decisions, plans, validation reports, execution reports, and audit traces are persisted as structured records. This supports offline analysis, report generation, and comparison against baselines.

### 3.5 Kubernetes Compatibility

The prototype integrates with Kubernetes deployments, the Kubernetes Python client, Prometheus, and workload manifests for experiment scenarios.

## 4. System Overview

The controller follows this pipeline:

`monitor -> analyze -> plan -> safety -> execute -> knowledge -> reasoning`

The implementation is organized under `src/adaptive_controller`:

| Layer         | Package                                        | Responsibility                                                                            |
|---------------|------------------------------------------------|-------------------------------------------------------------------------------------------|
| Monitor       | <code>adaptive_controller.monitor</code>       | Collect service metrics from Prometheus.                                                  |
| Analyze       | <code>adaptive_controller.analyze</code>       | Detect threshold violations, SLA issues, readiness problems, missing signals, and trends. |
| Plan          | <code>adaptive_controller.plan</code>          | Generate scale-up, scale-down, no-op, or future configuration-change plans.               |
| Safety        | <code>adaptive_controller.safety</code>        | Validate plans against hard constraints.                                                  |
| Execute       | <code>adaptive_controller.execute</code>       | Apply approved Kubernetes deployment scaling actions.                                     |
| Knowledge     | <code>adaptive_controller.knowledge</code>     | Persist metrics, plans, validations, executions, policies, and scenario context.          |
| Reasoning     | <code>adaptive_controller.reasoning</code>     | Generate explanations from read-only decision context.                                    |
| Baselines     | <code>adaptive_controller.baselines</code>     | Provide HPA-style, PID, and rule-based comparison controllers.                            |
| Observability | <code>adaptive_controller.observability</code> | Emit traces, decision logs, and audit logs.                                               |

The `ControlLoop` object coordinates these components and returns a structured `ControlLoopResult`.

## 5. MAPE-K Implementation

### 5.1 Monitor

The monitor layer queries Prometheus for service-level signals:

- CPU usage.
- Memory usage.
- Request rate.
- Error rate.
- p95 latency.
- Desired replicas.
- Current replicas.
- Ready pods.

Unavailable metrics are represented explicitly rather than causing the loop to crash. This is important because production observability is imperfect. A controller that cannot distinguish between “healthy” and “unknown” can easily make unsafe decisions.

## 5.2 Analyze

The analyzer converts metric snapshots into findings. It detects:

- High CPU usage.
- High memory usage.
- p95 latency SLA violations.
- Error-rate violations.
- Low throughput.
- Missing metrics.
- Pod readiness issues.
- Trend changes.

The analyzer is intentionally deterministic. It does not ask an LLM whether the system is healthy. This preserves reproducibility and makes test coverage meaningful.

## 5.3 Plan

The planner maps findings to candidate adaptation plans. Current actions include:

- `scale_up`
- `scale_down`
- `no_op`
- `config_change` as a future extension point

An adaptation plan includes a service name, action, priority, reason, current replica count, target replica count, confidence, source findings, and metadata.

The planner proposes; it does not execute.

## 5.4 Safety

The safety validator is the central control boundary. It approves or rejects plans using hard constraints:

- Scaling plans must include current and target replica counts.
- Target replicas must not fall below the configured minimum.
- Target replicas must not exceed the configured maximum.
- Total replicas must remain within the configured cluster budget.
- Estimated cost must remain within budget.
- Services must respect a cooldown window.

Rejected plans include guard-specific reasons. This allows the reasoning and audit layers to explain not only what was rejected, but why.

## 5.5 Execute

The execution layer applies only safety-approved plans. The current implementation supports Kubernetes deployment scaling through the scale subresource:

```
patch_namespaced_deployment_scale(namespace, deployment, replicas)
```

The default command-line entry point runs in dry-run mode. Real mutation requires `execute_actions=True` and a configured deployment executor.

## 5.6 Knowledge

The knowledge layer stores structured records for:

- Metrics.
- Plans.
- Validation results.

- Execution reports.
- Policies.
- Scenarios.
- Retrieved context.

The current prototype uses JSONL-backed storage, which is useful for research and offline analysis. A production system would likely replace or supplement this with durable database storage or Kubernetes custom resource status.

## 5.7 Reasoning

The reasoning layer provides explanations. By default, it uses an offline deterministic explanation generator so tests and experiments remain reproducible.

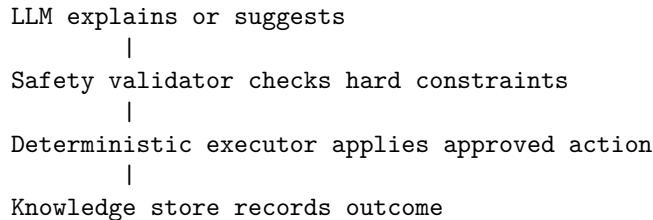
If real LLM support is introduced, the LLM should receive read-only context:

- Analysis report.
- Plan batch.
- Validation report.
- Retrieved knowledge.

The LLM response should be used for explanation only. It should not become an executable plan and should not override deterministic safety checks.

## 6. Safety Architecture

The most important design decision is the separation between reasoning and actuation.



This separation addresses a key risk in AI-assisted operations: the temptation to let a model directly decide and execute changes. In this prototype, all executable actions are produced by deterministic planning code and validated by deterministic guards. The reasoning layer can make the system easier to understand, but not more permissive.

This boundary supports:

- Auditability.
- Reproducibility.
- Operator trust.
- Testable safety constraints.
- Controlled experimentation with LLM providers.

## 7. Prototype Capabilities

The repository currently includes:

- Python package for the adaptive controller.
- Prometheus monitoring integration.
- Deterministic analyzers and planners.
- Safety validation guards.
- Kubernetes execution adapter.
- JSONL knowledge and observability logs.

- Offline reasoning and explanation layer.
- HPA-style, PID, and rule-based baselines.
- Sock Shop workload manifests and load tests.
- Lightweight Online Boutique manifests and load tests.
- Experiment runner utilities.
- Evaluation metric helpers.
- Statistical comparison helpers.
- Report generation utilities.
- Kubernetes manifests for namespaces, controller deployment, RBAC, HPA, Prometheus, and Grafana.

The prototype is useful as:

- A research artifact for self-adaptive systems.
- A teaching example for MAPE-K architectures.
- A starting point for custom Kubernetes autoscaling experiments.
- A blueprint for safe AI-assisted operations.
- A comparison harness for adaptive and baseline controllers.

## 8. Example Usage

### 8.1 Local Dry Run

Install dependencies:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Run one dry-run control-loop iteration:

```
python -m adaptive_controller.main
```

### 8.2 Experiment Run

Run the adaptive controller experiment:

```
python experiments/runners/run_experiment.py experiments/configs/llm_adaptive_controller.yaml
```

Run baseline controller setup:

```
python experiments/runners/run_baseline.py experiments/configs/hpa_baseline.yaml
python experiments/runners/run_baseline.py experiments/configs/pid_controller.yaml
```

Collect result summaries:

```
python experiments/runners/collect_results.py experiments/results/raw --output experiments/results/summ
```

### 8.3 Kubernetes Deployment

Create a local cluster:

```
./scripts/setup_cluster.sh
```

Deploy monitoring:

```
./scripts/deploy_monitoring.sh
```

Deploy workloads:

```
./scripts/deploy_workloads.sh
```

Deploy the controller:

```
./scripts/deploy_controller.sh
```

## 9. Evaluation Methodology

The prototype is designed to support comparative experiments rather than a single anecdotal demonstration.

### 9.1 Controllers Compared

The evaluation framework supports comparison among:

- Adaptive controller.
- HPA-style baseline.
- PID controller.
- Rule-based controller.

### 9.2 Workload Scenarios

The repository includes load-test patterns for:

- Steady-state traffic.
- Flash-crowd traffic.
- Gradual-ramp traffic.

These scenarios can be applied to Sock Shop or the lightweight Online Boutique workload.

### 9.3 Metrics

Suggested evaluation metrics include:

- p95 latency.
- SLA violation count or duration.
- Error rate.
- Request throughput.
- Average replica count.
- Peak replica count.
- Scaling frequency.
- Adaptation latency.
- Oscillation rate.
- Resource cost estimate.
- Safety rejection count.
- Explanation availability.

### 9.4 Experimental Procedure

A reproducible experiment should:

1. Deploy the workload and monitoring stack.
2. Select one controller configuration.
3. Apply one workload pattern.
4. Collect metrics, decisions, validation reports, execution reports, and load-test summaries.
5. Repeat for each baseline.
6. Repeat trials enough times to support statistical analysis.
7. Compute evaluation metrics.
8. Generate comparison tables and charts.

## 9.5 Statistical Analysis

The repository includes helpers for:

- Paired t-statistics.
- Confidence intervals.
- Effect size.
- Bonferroni correction.

Repeated trials are required before making strong empirical claims about performance improvement.

## 9.6 Research Questions

The prototype can support the following questions:

- Does the adaptive controller reduce SLA violations compared with baselines?
- Does it avoid unnecessary scaling?
- Does safety validation reduce unstable or unsafe actions?
- Does the reasoning layer improve explainability without compromising deterministic control?
- How often are proposed plans rejected by safety guards?
- What is the tradeoff between cost and latency under different workload patterns?

## 10. Production Readiness Gap

The prototype is not yet a production-ready autoscaler. It demonstrates an architecture and research direction. A production deployment would require additional engineering:

- Kubernetes-native reconciliation loop.
- Leader election.
- Durable state management.
- Stronger failure handling.
- Full observability metrics and dashboards.
- Policy versioning and validation.
- Human approval workflows.
- Rollback and post-execution verification.
- Concurrency control with HPA, GitOps, and manual operations.
- Security hardening and least-privilege RBAC.
- Container image build and release pipeline.
- End-to-end tests in real clusters.
- Long-running load and failure experiments.
- LLM governance, prompt versioning, and redaction.

These items are expanded in `NEXT_STEPS.md`.

## 11. Limitations

This prototype has several important limitations.

First, the current implementation focuses primarily on deployment scaling. Other adaptation actions, such as configuration changes, traffic routing, circuit breaking, or vertical scaling, are not fully implemented.

Second, the knowledge layer uses JSONL storage. This is appropriate for experiments but insufficient for production-grade durability, queryability, and concurrent access.

Third, the default reasoning mode is offline and deterministic. This is intentional for reproducibility, but it means the prototype does not yet evaluate real LLM provider behavior under operational constraints.

Fourth, the project currently provides an evaluation framework, but publishable empirical claims require repeated experiments, controlled environments, and reported quantitative results.



Fifth, production-grade Kubernetes controllers require reconciliation semantics, leader election, health probes, robust retry behavior, and careful interaction with existing controllers such as HPA and GitOps agents.

## **12. Future Work**

Future work should proceed in four phases.

### **12.1 Production-Safe Dry Run**

Run the controller continuously in Kubernetes without mutation. Add self-metrics, structured logs, health probes, policy loading, and decision status reporting.

### **12.2 Guarded Execution**

Enable low-risk scaling actions with pre-checks, post-checks, persistent cooldown state, rollback behavior, and hardened RBAC.

### **12.3 Operator Workflow**

Add human approval, freeze windows, decision history dashboards, explanation review, and incident annotations.

### **12.4 Advanced Adaptation**

Extend the planner toward multi-objective optimization across latency, cost, stability, availability, and service priority. Introduce controlled LLM provider integrations with prompt governance and redaction.

## **13. Practical Value**

A software engineering team could use this work in several ways.

### **13.1 Reference Architecture**

The project demonstrates how to structure adaptive control around explicit phases and typed artifacts rather than an opaque automation script.

### **13.2 Experimental Harness**

Researchers can compare adaptive strategies against HPA-style, PID, and rule-based baselines using repeatable workload patterns.

### **13.3 Safe AI-Assisted Operations Pattern**

The design provides a practical pattern for using LLMs in operations without granting them direct infrastructure authority.

### **13.4 Custom Autoscaling Foundation**

Platform teams can adapt the deterministic planner and safety validator to build service-specific scaling policies that consider latency, throughput, error rate, readiness, and cost.

### **13.5 Teaching and Demonstration**

The repository can be used to teach MAPE-K, Kubernetes autoscaling, safety validation, and explainable runtime adaptation.

## 14. Conclusion

Self-adaptive microservice systems need more than reactive scaling. They need trustworthy control loops that observe meaningful signals, reason about service health, validate proposed changes, execute safely, and explain decisions to humans. This prototype shows one way to combine deterministic safety with AI-assisted reasoning while preserving a hard boundary between explanation and actuation.

The central contribution is architectural: AI can support explainability and context awareness without becoming the actuator. By keeping planning, validation, and execution deterministic, the system remains testable, auditable, and reproducible. This makes the prototype a useful research foundation for safe adaptive control in Kubernetes environments and a practical starting point for future production-grade adaptive platforms.

## References

1. Kubernetes Documentation. “Horizontal Pod Autoscaling.” <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
2. Prometheus Documentation. “Overview.” <https://prometheus.io/docs/introduction/overview/>
3. White, S. R., Hanson, J. E., Whalley, I., Chess, D. M., Kephart, J. O., and Segal, A. “Autonomic Computing: An Architectural Approach and Prototype.” *Integrated Computer-Aided Engineering*, 2006. <https://journals.sagepub.com/doi/pdf/10.3233/ICA-2006-13206>
4. Cheng, B. H. C., de Lemos, R., Giese, H., Inverardi, P., Magee, J., Andersson, J., Becker, B., Bencomo, N., Brun, Y., Cukic, B., Di Marzo Serugendo, G., Dustdar, S., Finkelstein, A., Gacek, C., Geihs, K., Grassi, V., Karsai, G., Kienle, H. M., Kramer, J., Litoiu, M., Malek, S., Mirandola, R., Muller, H. A., Park, S., Shaw, M., Tichy, M., Tivoli, M., Weyns, D., and Whittle, J. “Software Engineering for Self-Adaptive Systems: A Research Roadmap.” 2009. <https://people.cs.umass.edu/~brun/pubs/pubs/Cheng09.pdf>